

LegalEase

AI-Powered Legal Document Generator

PROJECT IMPLEMENTATION & SUBMISSION DOCUMENT

BACHELOR OF COMPUTER APPLICATIONS (BCA)

Academic Year: 2026-2027

12. Project Modules

13. Testing Plan

14. Expected Outputs

15. Security and Responsible Use

16. Future Enhancements

17. Conclusion

18. References

19. Final Submission Checklist

1. PROJECT DESCRIPTION

LegalEase leverages Generative AI to simplify the creation of legal documents by providing customizable, accurate and editable templates for a wide range of use cases. Users can generate employment contracts, lease agreements, NDAs and other documents tailored to inputs such as involved parties, effective dates and key terms.

The supplied specification defines a FastAPI backend, a Streamlit frontend and a Google Gemini AI core. The application is designed to maintain document formatting standards, support branding such as logos and fonts, provide editable previews and offer export options in PDF, DOCX and TXT formats.

The platform is intended as an AI-assisted drafting tool. Generated legal text should be reviewed appropriately before actual legal use.

2. PROJECT SCENARIOS

Scenario 1 – Employment Contract

A startup founder enters details about a new hire and selects "Employment Contract". LegalEase generates a structured agreement containing roles, responsibilities, compensation and confidentiality clauses. The user can customize the wording, add a company logo and download a branded PDF.

Scenario 2 – Non-Disclosure Agreement

A freelancer enters the parties, scope of confidentiality and effective date to generate a structured NDA.

Scenario 3 – Residential Lease Agreement

A landlord enters the property address, tenant details and lease terms. LegalEase produces a formatted document with editable sections and a downloadable DOCX version.

3. PRE-REQUISITES

FastAPI framework knowledge – FastAPI Documentation.

Google Gemini API familiarity – Google Generative AI documentation.

Streamlit for frontend – Streamlit documentation.

Python programming proficiency – Python official documentation.

Development environment with Python 3.10+ and VS Code.

Specified package setup: `pip install fastapi uvicorn streamlit python-docx fpdf Pillow requests google-generativeai python-dotenv`

4. PROJECT WORKFLOW

The project is organized into five milestones: model selection and architecture; core functionality development; API logic integration; frontend development; and deployment/testing.

User Input → Streamlit → FastAPI → GeminiDocumentGenerator → Gemini AI → Generated Text → Preview/Edit → TXT / DOCX / PDF

5. MILESTONE 1 – MODEL SELECTION AND ARCHITECTURE

Activity 1.1 – Research and Select the Appropriate Model

Understand requirements: document type, parties involved, terms and conditions, and effective dates.

Explore Gemini documentation and the specified Gemini model approach.

Evaluate performance for structured prompting, long-context requirements, speed, formatting and reasoning.

Use the selected Gemini model for drafting formal documents with clear sections and legal terminology, as specified in the project brief.

Activity 1.2 – Define Architecture

Frontend: Streamlit for user interaction.

Backend: FastAPI application.

AI Core: Gemini integration.

Formatting modules: DOCX, PDF and TXT generators.

Frontend functions: text inputs for document type and dates; text areas for parties and terms; Generate Document; Edit Document; Download as TXT/DOCX/PDF; styled HTML preview.

Backend responsibilities: expose POST /generate; accept JSON input; route to GeminiDocumentGenerator; return structured text AI output.

Activity 1.3 – Set Up Development Environment

Install Python 3.10+.

Create virtual environment: `python -m venv venv`.

Install FastAPI and Uvicorn.

Install Streamlit.

Install Gemini SDK.

Install supporting libraries: `python-docx`, `fpdf`, `Pillow`, `requests`, `python-dotenv`.

Create the project structure.

Required project structure:

6. MILESTONE 2 – CORE FUNCTIONALITIES DEVELOPMENT

Activity 2.1 – Develop Core Functionalities

Legal Document Generation: generate contracts and agreements from user inputs such as document type, parties, terms and dates.

Document Output Formatting: `format_docx(text, doc_type)`, `format_pdf(text, doc_type)`, and `format_html_preview(text)`.

DOCX formatting: use `python-docx` with logo, Times New Roman, terms table and footer as specified.

PDF formatting: branded PDF with logo and footer.

HTML preview: styled preview displayed through Streamlit.

User interaction: collect user preferences through the Streamlit interface.

Editable document: provide an editable text area.

Download: provide TXT, DOCX and PDF outputs.

Activity 2.2 – Implement FastAPI Backend

Define the FastAPI /generate route. Process user input through a Pydantic DocumentRequest containing document_type, parties, terms and dates. Integrate the Gemini API through GeminiDocumentGenerator.generate_document().

Prompt structure specified in the brief: generate a comprehensive legal document titled {document_type}; involved parties: {parties}; effective date: {dates}; terms: {terms}; ensure formal legal structure.

7. MILESTONE 3 – API LOGIC INTEGRATION

Activity 3.1 – Create and Configure main.py

Initialize the FastAPI application, define the root endpoint to verify service availability and include routes from routes.py. The supplied example uses FastAPI(title='LegalEase') and a root response indicating that the LegalEase API is running.

Activity 3.2 – Handle Document Generation Logic in routes.py

Define the Pydantic DocumentRequest model to validate incoming data, initialize GeminiDocumentGenerator from ai_core.gemini_generator, create the POST /generate endpoint and return generated legal content.

8. MILESTONE 4 – FRONTEND DEVELOPMENT

Activity 4.1 – Designing UI

Configure the Streamlit page with page title "LegalEase" and a centered layout.

Create a header and centered title: "LegalEase – AI Legal Document Generator".

Create the user-input interface for document_type, parties, terms and dates.

Activity 4.2 – Dynamic Previews and Export

Generate Document: send a POST request to the FastAPI /generate endpoint.

Sanitize generated text before rendering.

HTML Preview: display an elegant dark-themed semantic HTML preview.

Editable Preview: show generated text in an editable text area.

Multi-format Download: provide TXT, DOCX and PDF outputs.

DOCX formatting includes logo, headings, table and footer; PDF formatting includes branding and footer; HTML preview is styled for web display.

9. MILESTONE 5 – DEPLOYMENT

Activity 5.1 – Preparing for Local Deployment

Activate the virtual environment: venv\Scripts\activate on Windows, or source venv/bin/activate on Mac/Linux.

Install dependencies using pip install -r requirements.txt.

Create the .env file and configure GEMINI_API_KEY=your_api_key_here.

Activity 5.2 – Testing and Verifying Local Deployment

Launch the FastAPI backend using the specified Uvicorn command.

Verify the backend at http://127.0.0.1:8000.

Launch the Streamlit frontend using streamlit run frontend/app.py.

Open the local Streamlit URL, such as http://localhost:8501.

Enter document type, parties, terms and date.

Press Generate Document.

Check preview, edit text and download TXT/DOCX/PDF.

Check logo/footer placement, bullet formatting and embedding of terms.

10. EXPLORING LEGALEASE – INPUT GUIDE

10.1 Document Type

Purpose: choose the type of legal document to generate. Examples specified in the brief: Agreement, Contract, NDA, Lease Agreement and Employment Offer Letter. Example input: Freelance Work Contract.

10.2 Parties Involved

Purpose: provide names and roles. Examples include John Doe (Freelancer), ABC Corp (Client), Alice Smith (Tenant), XYZ Realty (Landlord), Jane Doe (Service Provider) and TechNova Inc. (Client).

10.3 Terms & Conditions

Use semicolons for bullet points. Examples include payment within 30 days of invoice, delivery by an agreed deadline, confidentiality requirements and termination with notice.

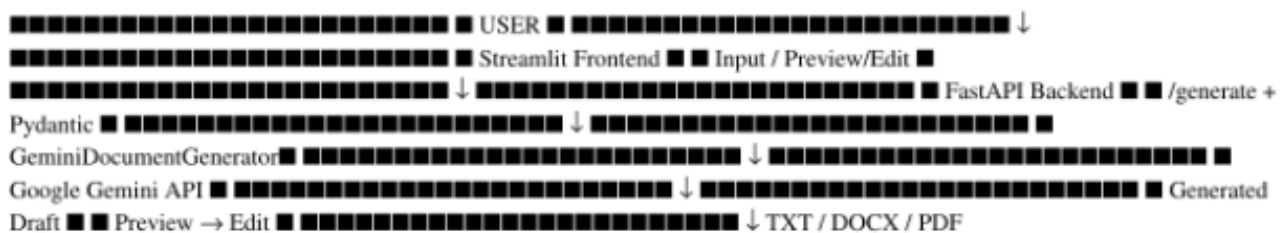
10.4 Effective Date

Purpose: date when the agreement becomes effective. Examples specified: April 10, 2025; 10/04/2025; April 15, 2025.

10.5 Editing and Downloading

The specified workflow opens an edit window through “Click to Edit Document”. TXT is the simple text output; DOCX contains formatted Word content with branding elements; PDF is the formatted branded output.

11. SYSTEM ARCHITECTURE



12. PROJECT MODULES

Frontend Module – Streamlit-based interface for input, generation, preview, editing and downloads.

Backend Module – FastAPI application and API routing.

AI Core Module – Gemini API integration and structured prompt generation.

DOCX Module – formatted Word document generation.

PDF Module – branded PDF generation.

TXT Module – plain-text export.

Preview Module – styled HTML preview of generated content.

Configuration Module – .env-based API key configuration.

13. TESTING PLAN

14. EXPECTED OUTPUTS

Structured AI-generated first draft based on supplied inputs.

Editable document preview.

TXT download.

DOCX download with specified formatting elements.

PDF download with specified branding elements.

FastAPI backend response through /generate.

15. SECURITY AND RESPONSIBLE USE

Keep the Gemini API key in .env and do not expose it in frontend code.

Do not place real confidential legal information in screenshots or demonstration examples.

Validate and sanitize user inputs before processing.

Treat generated content as a draft and review it before legal use.

Use appropriate privacy and security practices when handling sensitive information.

16. FUTURE ENHANCEMENTS

Multi-jurisdiction support.

Multi-language document generation.

Voice-assisted data entry.

Document comparison and change tracking.

Secure digital-signature integration.

Advanced document classification.

Retrieval-based reference to approved legal resources.

Role-based access and stronger audit logging.

Mobile application support.

Integration with professional legal-review workflows.

17. CONCLUSION

LegalEase represents an AI-assisted approach to simplifying the initial creation and organization of legal documents. The supplied project specification combines a Streamlit user interface, FastAPI backend, Gemini AI integration and document-formatting components to provide a guided workflow from user input to editable and downloadable output.

The project demonstrates how generative AI can support structured drafting while keeping the user involved in reviewing and editing the result. The application is designed to improve accessibility and reduce repetitive first-draft preparation work, while recognizing that generated legal text can require appropriate review before actual legal use.

18. REFERENCES

Google Gemini / Google Generative AI Documentation.

FastAPI Documentation.

Streamlit Documentation.

Python Official Documentation.

python-docx documentation.

FPDF documentation.

Pillow documentation.

python-dotenv documentation.

19. FINAL SUBMISSION CHECKLIST

■ Project folder created with the specified structure.

■ Python environment configured.

■ Required packages installed.

■ Gemini API configuration completed.

■ FastAPI backend tested.

■ Streamlit frontend tested.

■ Document generation tested.

■ Editing function tested.

- TXT export tested.
- DOCX export tested.
- PDF export tested.
- Screenshots captured from the actual working project.
- Student details and guide details filled in.
- Final report reviewed and formatted before submission.

APPENDIX A – SCREENSHOT INSERTION PLAN

Figure 1 – Project folder structure in VS Code [INSERT ACTUAL SCREENSHOT HERE]

Figure 2 – .env / configuration setup (hide the actual API key) [INSERT ACTUAL SCREENSHOT HERE]

Figure 3 – FastAPI running successfully [INSERT ACTUAL SCREENSHOT HERE]

Figure 4 – LegalEase Streamlit home page [INSERT ACTUAL SCREENSHOT HERE]

Figure 5 – User input form [INSERT ACTUAL SCREENSHOT HERE]

Figure 6 – Generated document preview [INSERT ACTUAL SCREENSHOT HERE]

Figure 7 – Edit Document interface [INSERT ACTUAL SCREENSHOT HERE]

Figure 8 – TXT/DOCX/PDF download options [INSERT ACTUAL SCREENSHOT HERE]

Figure 9 – Generated DOCX output [INSERT ACTUAL SCREENSHOT HERE]

Figure 10 – Generated PDF output [INSERT ACTUAL SCREENSHOT HERE]